

Supplemental Material: Beyond Task Completion: Auditing Evolving Tool Libraries in Agentic AI

Anonymous Author(s)

This supplement provides reproducibility material for the strict verified-subset pilot reported in the main paper. All tables and manifest files are regenerated from `results_canonical/` by the scripts described in Section 4 of the main paper (`build_canonical.py`, `stats_report.py`, `make_tables.py`, `make_figures.py`).

1 Reproducibility Summary

Model and decoding. All strict-pilot runs use Claude Haiku 4.5 (snapshot `claude-haiku-4-5-20251001`), accessed via the Anthropic API with temperature 1.0 and the model default top- p . Tool execution is sandboxed in-process with a five-second wall-clock limit; harness state resets between sessions so library accumulation is per-session.

Seeds and sessions. Three integer seeds ($0, 1, 2$) $\times$ eight verified-subset sessions $\times$ five protocols yields 120 session rows. Each session contributes a paired observation per protocol in the contrast analyses.

Statistics. Per-session paired differences are bootstrap-resampled $B=10,000$ times with seed 20260604; 95% confidence intervals are percentile CIs over the bootstrap distribution; the three pre-specified contrasts (Table 3 in the main paper) are Benjamini–Hochberg adjusted.

Canonical artifacts. The release includes the canonical run manifest (`run_manifest.jsonl`, 120 rows), tool manifest (`tool_manifest.jsonl`, system-level summary), aggregate (`aggregate.json`), audit report (`audit_report.json`), statistical report (`statistical_report.json`), and the pre-specified contrasts (`claims.json`). Every cell in the main-paper tables can be derived from these files by the scripts above.

2 Audit Coverage Detail

Table 1 shows the per-session decomposition of the strict subset. Across the eight verified-subset sessions, 51 graded task decisions per system pass survive into the strict TC denominator. Seed tasks are explicitly excluded from the denominator by design: they exist to expose the agent to the seed library before any creation and carry no graded outcome.

3 Baseline Fairness Ledger

Three of the five protocols evaluated in the strict pilot are adaptations of prior published systems to a shared open-task harness; No-Evolution and One-Shot are native to this harness. Table 2 lists what was preserved from each original design and where the adaptation deviates. The `-style` suffix

Table 1: Per-session strict coverage. Columns are role-level verified/total. Seed counts are reported but excluded from TC (no graded outcome).

Session	Seed	Gap	Var.	Comp.	Reg.	Adv.
data_transform_s1	0/3	2/2	2/2	0/1	1/1	1/2
data_transform_s2	0/3	2/2	2/2	1/1	1/1	1/2
data_transform_s3	0/3	2/2	2/2	1/1	1/1	1/2
data_transform_s4	0/3	2/2	2/2	1/1	1/1	2/2
data_transform_s5	0/3	2/2	2/2	1/1	1/1	2/2
numerical_s1	0/3	2/2	2/2	0/1	1/1	1/2
numerical_s2	0/3	1/2	1/2	0/1	1/1	1/2
numerical_s3	0/3	2/2	2/2	0/1	1/1	0/2
Strict subset	0/24	15/16	15/16	4/8	8/8	9/16

on EvoSkill-style, ToolMaker-style, and CREATOR-style is deliberate: no underperformance in this pilot should be read as a verdict on the original methods.

Protocol	Native setting	Preserved faithfully
EvoSkill-style	Skill-library agent with NL strategy proposer/prompt generator	Skill-proposer and prompt-templates; strategy-folder
CREATOR-style	Decision/execution split tool creation, with native reference tests	Two-phase decide-then-control flow; per-tool synthesis
ToolMaker-style	Repo-grounded with reference tests; diagnose-and-rewrite loop gated by provided tests	Two-stage diagnose-the-prompt structure; sandbox of auto-tests

Table 2: Baseline fairness ledger for adapted protocols. All systems share the identical task stream, seed library, model (Claude Haiku 4.5), decoding parameters, and verifier set; only the tool-creation control flow varies.

4 Worked Trace Example: a Self-Validating Tool that Fails Hidden Tests

The main paper points to a worked case in `data_transform_s1` where the audit layer flags a tool that TC scores as successful. This appendix expands the case.

The shared capability `abr_binary_decode` appears in five session-1 tasks: `gap_1` (which invites creation of the capability), `variant_1`, `compose_1`, `regress_1`, and `adversarial_1` (each of which can reuse the tool created at `gap_1`). In the preserved-source Sonnet pilot, a single

tool (`decode_abr_format`) was synthesised alongside an agent-written test file. The tool passed its self-tests at $C=0.50$; the session-level TC reached 0.95.

When the same tool source is replayed against the held-out conformance suite tied to the capability (six unit inputs and six adversarial inputs, none seen by the agent), correctness is 0.00 and robustness/generality are 0.50/0.00. The tool is implementing a parser that structurally agrees with its own training-style examples but does not implement the capability’s contract.

What the audit layer surfaces.

- Session TC: 0.95 – a near-perfect session.
- In-session reuse: 1 correct, 1 incorrect – ambiguous on its own.
- Hidden-test correctness on the same artifact: 0.00 – the tool fails every held-out input for the capability it claims to implement.

The header pattern (TC high, in-session reuse mixed, hidden-test correctness near zero on the underlying artifact) is exactly the failure mode the framework is designed to expose. Per-tool conformance testing under the strict pilot regime is the next artifact-hardening step (see Limitations in the main paper).

5 Auxiliary Signal: Library-Health Contrasts under a Sonnet Pilot

A separate preserved-source Sonnet pilot (four seeds, nine sessions) applies the same harness, manifests, and verifier set as the verified-subset Haiku pilot in the main paper. Per-tool source preservation makes the capability-aligned conformance layer computable, so library health (LH) – combining correct reuse, redundancy, utilisation, composition, regression, and the hidden-test quality gate – can be measured at the artifact level. This is the same audit schema as the main pilot; only the model snapshot and the source-preservation setting differ, which makes the contrasts here a same-schema-different-model probe of the framework’s discriminative power, not a TC claim against the verified-subset pilot.

Internal manifest labels. The released `statistical_report.json` uses the project’s internal names; the table below uses the public naming from the main paper. The mapping is: `arise` → “Code-Evol” (an internal preserved-source pilot lineage); `evoskill` → “Strategy-Only” in the LH analysis because LH does not depend on whether the strategy was executed; `creator-style` and `no-evolution` keep the same labels.

These contrasts show that the framework can distinguish tool-creating systems from non-creating systems on a measurable artifact dimension (LH) when the underlying tool source is preserved and per-tool conformance is aligned. Achieving the same alignment under the strict Haiku pilot is the next

Table 3: Library-health contrasts in the Sonnet pilot (Sonnet, $n=4$ seeds $\times$ 9 sessions; per-tool source preserved). Paired hierarchical bootstrap over (seed, session), 10 000 resamples; BH-adjusted bootstrap p . Generated by `scripts/stats_report.py`.

Contrast (LH)	Δ	95% CI	p_{adj}
Code-Evol – No-Evol	12.0*	[6.8, 17.0]	0.000
CREATOR-style – No-Evol	15.2*	[8.0, 23.4]	0.000
Code-Evol – Strategy-Only	10.4*	[6.4, 14.7]	0.000
CREATOR-style – Strategy-Only	13.6*	[7.5, 21.3]	0.000

*95% CI excludes 0. LH in percentage points.

artifact-hardening step. The absolute LH levels and the magnitudes of these contrasts are provider-specific and should not be transferred to the Haiku strict pilot.

A Artifact Gallery

The benchmark preserves every synthesised tool to disk under `results_full/<config>/tools/<session>/<name>.py` (plus a `.meta.json` with the per-tool TQS scores). This appendix presents a side-by-side gallery of representative tools from the two tool-creating systems with multi-seed preserved source — Code-Evol (semantic- Q run) and CREATOR-style — to make the per-tool failure modes the benchmark surfaces concrete. Per-dimension scores are shown inline (C , R , G , Q and the composite TQS); bold indicates ≥ 0.5 .

A.1 Code-Evol/Sonnet (semantic Q)

15 tools were synthesised and preserved on disk across 9 sessions. Below we show 5 representative tools spanning the quality spectrum: correct exemplars, fragile high- Q /low- C tools, duplicates, and trivial (low-TQS) cases.

Correct exemplar (*passes hidden tests with $C \geq 0.5$*)

`http_get_with_headers` (*api_orchestration.s1*) $C=0.50$ $R=0.50$ $G=0.50$ $Q=1.00$ TQS=**0.62**

```
def http_get_with_headers(url: str, headers: dict) -> str:
    """Make an HTTP GET request with custom headers."""
    import urllib.request
    import urllib.error

    try:
        # Create request object with custom headers
        request = urllib.request.Request(url)
        # ...
    except Exception as e:
        return f"Error: {str(e)}"
```

Correct exemplar (*passes hidden tests with $C \geq 0.5$*)

`compute_stats` (*numerical.s1*) $C=0.50$ $R=0.50$ $G=0.50$ $Q=1.00$ TQS=**0.62**

```
def compute_stats(data: list[float]) -> dict[str, float]:
    """Calculate basic statistical measures for a numerical dataset."""
    import math

    try:
        if not data:
            return {"error": "Empty dataset provided"}
        # ...
    except Exception as e:
        return {"error": f"Computation failed: {str(e)}"}
```

Fragile (*high Q , low C – looks well-formed, fails on hidden inputs*)

`hmac_sha256` (*api_orchestration.s1*) $C=0.00$ $R=0.50$ $G=0.00$ $Q=1.00$ TQS=0.38

```
def hmac_sha256(secret: str, message: str) -> str:
    """Generate HMAC-SHA256 hash for authentication and security purposes."""
    import hmac
    import hashlib

    try:
        # Handle None inputs explicitly
        if secret is None or message is None:
            # ...
    except Exception as e:
        return f"Error generating HMAC-SHA256: {str(e)}"
```

Fragile (*high Q , low C – looks well-formed, fails on hidden inputs*)

`compute_crc32` (*data_transform.s5*) $C=0.00$ $R=1.00$ $G=1.00$ $Q=1.00$ TQS=**0.75**

```
def compute_crc32(data: bytes) -> int:
    """Compute CRC32 checksum for data integrity verification."""
    import struct

    try:
        if not isinstance(data, bytes):
            return -1
        # ...
    except Exception:
        return -1
```

Duplicate (*synthesised across multiple sessions*)

`decode_base64` (*data_transform.s5*) $C=0.50$ $R=0.50$ $G=0.50$ $Q=1.00$ TQS=**0.62**

```
def decode_base64(encoded_string: str) -> bytes:
    """Decode base64 encoded binary data into bytes."""
    import base64
```

```

349     # Validate input type
350     if not isinstance(encoded_string, str):
351         raise TypeError(f"Expected string, got {type(encoded_string).__name__}")
352
353     # ...
354     # Return empty bytes on any decoding error (invalid base64, etc.)
355     return b''

```

A.2 CREATOR-style/Sonnet

18 tools were synthesised and preserved on disk across 9 sessions. Below we show 5 representative tools spanning the quality spectrum: correct exemplars, fragile high- Q /low- C tools, duplicates, and trivial (low-TQS) cases.

Correct exemplar (*passes hidden tests with $C \geq 0.5$*)
decode_guardian_minimal_data (*data_transform_s5*) $C=0.50$ $R=0.50$ $G=0.50$ $Q=0.89$ TQS=0.60

```

361 def decode_guardian_minimal_data(base64_data):
362     """Decode minimal GUARDIAN data format handling single blocks and edge cases."""
363     import base64
364     import struct
365
366     try:
367         # Decode base64 data
368         raw_data = base64.b64decode(base64_data)
369
370     except Exception as e:
371         return {'text': '', 'integrity': False, 'blocks_processed': 0, 'details': f'Error: {str(e)}'}

```

Correct exemplar (*passes hidden tests with $C \geq 0.5$*)
decode_qlog_with_continuations (*data_transform_s3*) $C=0.50$ $R=0.50$ $G=0.50$ $Q=0.89$ TQS=0.60

```

372 def decode_qlog_with_continuations(base64_data):
373     """Decode QLOG data and merge continuation entries with their parent entries."""
374     import base64
375     import struct
376     from datetime import datetime, timezone
377
378     # Decode base64 data
379     binary_data = base64.b64decode(base64_data)
380     # ...
381
382     return merged_entries

```

Fragile (*high Q , low C – looks well-formed, fails on hidden inputs*)
fetch_all_users_with_pagination (*api_orchestration_s1*) $C=0.00$ $R=0.50$ $G=1.00$ $Q=0.89$ TQS=0.60

```

383 def fetch_all_users_with_pagination(base_url="http://127.0.0.1:18080/api/users"):
384     """Utility: Fetches all users from a paginated API endpoint using cursor-based pagination.
385     Continues fetching until has_more is False, collecting all user data across pages."""
386     import urllib.request
387     import urllib.parse
388     import json
389
390     all_users = []
391     cursor = None
392     # ...
393     'names': names
394 }

```

Fragile (*high Q , low C – looks well-formed, fails on hidden inputs*)
decode_qlog_binary_data (*data_transform_s3*) $C=0.00$ $R=0.50$ $G=0.00$ $Q=0.78$ TQS=0.32

```

394 def decode_qlog_binary_data(base64_data):
395     """Decode QLOG (Quantized Log Format) binary data into structured log records."""
396     import base64
397     import struct
398     from datetime import datetime, timezone
399
400     # Decode base64 data
401     binary_data = base64.b64decode(base64_data)
402     # ...
403
404     return records

```

Trivial (*low overall TQS*)
deserialize_tpack (*data_transform_s4*) $C=0.00$ $R=0.50$ $G=0.00$ $Q=0.67$ TQS=0.29

```

405 def deserialize_tpack(base64_data):
406     """Deserialize TPACK (Tagged Pack Format) binary data into Python objects."""

```

465		523
466	import base64	524
467	import struct	525
468	def parse_varint(data, offset):	526
469	result = 0	527
470	shift = 0	528
471	# ...	529
472	result, _ = parse_value(binary_data, 0)	530
473	return result	531
474		532
475		533
476		534
477		535
478		536
479		537
480		538
481		539
482		540
483		541
484		542
485		543
486		544
487		545
488		546
489		547
490		548
491		549
492		550
493		551
494		552
495		553
496		554
497		555
498		556
499		557
500		558
501		559
502		560
503		561
504		562
505		563
506		564
507		565
508		566
509		567
510		568
511		569
512		570
513		571
514		572
515		573
516		574
517		575
518		576
519		577
520		578
521		579
522		580